Control of a SCARA Manipulator

Technical Project - Foundations of Robotics

Prof. Bruno Siciliano

Elena d'Agostino
Vincenzo Esposito

Contents

1 Introduction **2**

2 Kinematic Model **2**
2.1 Frames Definition and Direct Kinematics Computation . . . 3
2.2 Jacobian Computation . . . 4
2.3 Reachable Workspace . . . 5

3 Manipulability Analysis **6**

4 Trajectory Planning **10**
4.1 Selection of Points . . . 10
4.2 Trajectory generation . . . 11

5 Inverse Kinematics Algorithms **14**
5.1 Jacobian Inverse . . . 15
5.2 Jacobian Transpose . . . 18
5.3 Jacobian Pseudo-Inverse . . . 20
5.4 Second-order algorithm . . . 24

6 Motion Control **27**
6.1 Dynamic Model . . . 27
6.2 Robust Control . . . 29
6.3 Adaptive Control . . . 33

1 Introduction

The main goal of this project is to analyze the so-called *SCARA* (Selective Compliance Assembly Robot Arm) manipulator from both kinematic and dynamic perspectives. To start with, in this introductory section, we first outline the main features of such a manipulator; a description of the work's structure is then provided.

As is well known, a SCARA manipulator describes a special geometry which can be realized by disposing two revolute joints, and one prismatic joint such that all the axes of motion in the manipulator are parallel (c.f. Figure 1). More specifically, a SCARA manipulator is characterized by the following elements:

- two revolute joints, which allows motion in a horizontal plane;
- one prismatic joint, enabling the third link to move vertically;
- one revolute joint, used for the orientation of the end-effector.

In such a configuration, the robot's joints are generally actuated by electric motors. Moreover, the correspondence of DOFs and Cartesian space variables is preserved solely for the vertical component of a task described in Cartesian coordinates; we also note that, the wrist positioning accuracy decreases as the distance between the wrist and the first joint axis grows.

This special geometry of a SCARA manipulator assures the robot to move freely and maintain stiffness in three axes while being compliant in the final axis. In particular, this feature makes it especially suited for high-precision and high-speed pick-and-place tasks, sorting, and assembly, particularly in electronic component manufacturing. For further details on the SCARA manipulator, see e.g. [1] and references therein.

In this project, our goal is to analyze the above defined structure from both kinematic and dynamic perspectives, plan its motion within a specified workspace, and implement well-known control algorithms to enable the manipulator to execute coordinated motions. The work is organized as follows: In the second section, we derive and discuss the Kinematic Model of the manipulator under examination; In section three a manipulability analysis is carried out, through a representation of both velocity and force manipulability ellipses, in relevant configurations for the robot manipulator; Planning of a desired trajectory for the robot to track in the operational space is realized in section four. Several simulation results of simulation of Closed Loop Inverse Kinematics Algorithms are presented and discussed in section five; Finally, in section six, the motion control problem is investigated, illustrating simulation results from the implementation of two notable control strategies, namely Robust Control and Adaptive Control.

2 Kinematic Model

This section deals with the kinematic model of a SCARA manipulator. We recall that, the main aim of direct kinematics is to compute the position and orientation of the end-effector frame with respect to the base frame of a robot manipulator. Quite generally, direct kinematics is described by means of a function $\boldsymbol{k}\colon \mathbb{R}^m \to \mathbb{R}$, such that, denoting by $\boldsymbol{x}_e$ the vector of operational space variables — i.e., the vector describing the end-effector position and orientation — we have:

$$\boldsymbol{x}_e = \boldsymbol{k}(\boldsymbol{q}), \tag{1}$$

where $\boldsymbol{q}$ is the vector of joint variables. The posture of the SCARA manipulator under examination is defined by 4 degrees of freedom (DOFs), as each joint provides a single degree of freedom, and is

described by the 4×1 vector of joint variables

$$\boldsymbol{q} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ d_3 \\ \theta_4 \end{bmatrix}. \tag{2}$$

θ_1, θ_2 and θ_4 are the angular displacement representing the relative rotations about axes of joints 1, 2 and 4 respectively, whereas d_3 denotes the linear displacement describing the relative translation along the axis of joint 3. We will articulate the different steps of the computation separately in the following subsections.

2.1 Frames Definition and Direct Kinematics Computation

As a first step in computing direct kinematics, we set the right-handed frames attached to each link according to the *Denavit-Hartenberg convention* (see Figure 1 below).

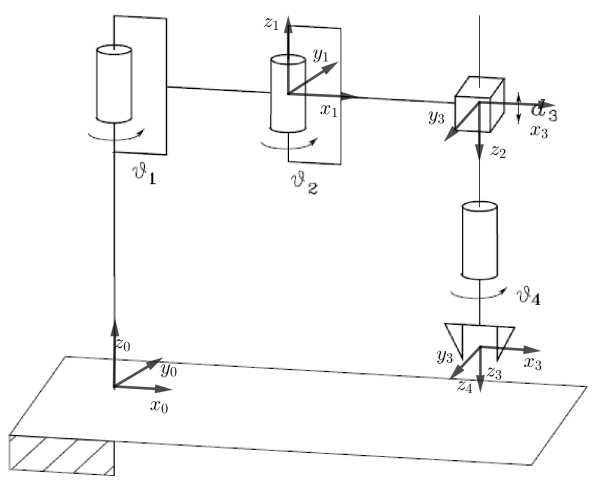

Figure 1: SCARA Manipulator.

According to the specific DH parameters (c.f. Table 1), we need to compute the transformation matrix representing the position and orientation of frame 4, fixed to the end-effector frame, with

respect to frame 0, that is,

$$\boldsymbol{T}_4^0(q) = \boldsymbol{A}_1^0(q_1)\boldsymbol{A}_2^1(q_2)\boldsymbol{A}_3^2(q_3)\boldsymbol{A}_4^3(q_4), \tag{3}$$

where

$$\boldsymbol{A}_i^{i-1} = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4}$$

represents the generic homogeneous transformation matrix of frame i with respect to frame $i-1$, for $i = 1, 2, 3, 4$.

Link	a_i	α_i	d_i	θ_i
1	0.5	0	1	θ_1
2	0.5	π	0	θ_2
3	0	0	d_3	0
4	0	0	0	θ_4

Table 1: Denavit-Hartenberg parameters for the SCARA manipulator.

Rather than relying our computation on transformation matrices (which shows an increasingly computational complexity), we can resort to the geometric method, which exploits the simplicity of the SCARA manipulator's structure. Specifically, the position and orientation of the end-effector can be expressed by means of the following law:

$$\boldsymbol{x}_e = \boldsymbol{k}(\boldsymbol{q}) = \begin{bmatrix} x \\ y \\ z \\ \phi \end{bmatrix} = \begin{bmatrix} a_1\cos(\theta_1) + a_2\cos(\theta_1+\theta_2) \\ a_1\sin(\theta_1) + a_2\sin(\theta_1+\theta_2) \\ d_0 - q_3 \\ \theta_1 + \theta_2 - \theta_4 \end{bmatrix}$$

where we set

- $\boldsymbol{q} = \begin{bmatrix}\theta_1 & \theta_2 & d_3 & \theta_4\end{bmatrix}^T$ for the vector of joint variables,
- a_1 and a_2 for the link lengths,
- d_0 for the fixed base height.

Hence, summarizing, we have the following relations between the parameters of interest

$$\begin{aligned} x &= a_1\cos(\theta_1) + a_2\cos(\theta_1+\theta_2) \\ y &= a_1\sin(\theta_1) + a_2\sin(\theta_1+\theta_2) \\ z &= d_0 - d_3 \\ \phi &= \theta_1 + \theta_2 - \theta_4 \end{aligned}$$

2.2 Jacobian Computation

Next, we switch to the Jacobian computation for the SCARA manipulator. We start by noting that the geometric Jacobian is given by

$$\boldsymbol{J}(\boldsymbol{q}) = \begin{bmatrix} \boldsymbol{z}_0 \times (\boldsymbol{p} - \boldsymbol{p}_0) & \boldsymbol{z}_1 \times (\boldsymbol{p} - \boldsymbol{p}_1) & \boldsymbol{z}_2 & \boldsymbol{z}_3 \times (\boldsymbol{p} - \boldsymbol{p}_3) \\ \boldsymbol{z}_0 & \boldsymbol{z}_1 & \boldsymbol{0} & \boldsymbol{z}_3 \end{bmatrix}, \tag{5}$$

where the various vectors can be computed from the manipulator direct kinematics. Therefore, the expression of the geometric Jacobian becomes

$$\boldsymbol{J}(\boldsymbol{q}) = \begin{bmatrix} -a_1\sin(\theta_1) - a_2\sin(\theta_1+\theta_2) & -a_2\sin(\theta_1+\theta_2) & 0 & 0 \\ a_1\cos(\theta_1) + a_2\cos(\theta_1+\theta_2) & a_2\cos(\theta_1+\theta_2) & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & -1 \end{bmatrix}. \tag{6}$$

In particular, matrix (6) above entails it is impossible to rotate about the x and y axes. Hence, whenever a four-dimensional operational space is considered, we can find the analytical Jacobian by simply eliminating rows 4 and 5 in matrix (6). Concluding, we arrive at the following expression of the analytical Jacobian

$$\boldsymbol{J}_A(\boldsymbol{q}) = \frac{\partial \boldsymbol{k}(\boldsymbol{q})}{\partial \boldsymbol{q}} = \begin{bmatrix} -a_1\sin(\theta_1) - a_2\sin(\theta_1+\theta_2) & -a_2\sin(\theta_1+\theta_2) & 0 & 0 \\ a_1\cos(\theta_1) + a_2\cos(\theta_1+\theta_2) & a_2\cos(\theta_1+\theta_2) & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 1 & 1 & 0 & -1 \end{bmatrix}. \tag{7}$$

In the next subsection, we will address the problem of the reachable workspace.

2.3 Reachable Workspace

Recall that a generic manipulator is characterized by joints and links of specific sizes. This then entails that not all points in the working environment can be reached by the end-effector. Therefore, as a starting point, we have to first define the so-called *reachable workspace*, namely, the region of the space the origin of the end-effector frame can occupy while achieving different orientations. Note that, the reachable workspace is limited, as there are mechanical constraints on the values of the joint variables. Precisely, we set

$$\begin{cases} \theta_1 \in [-\frac{\pi}{6}, \frac{7}{6}\pi], \\ \theta_2 \in [-\frac{5}{6}\pi, \frac{5}{6}\pi], \\ d_3 \in [0, 0.8]\, m, \\ \theta_4 \in [-\pi, \pi]. \end{cases} \tag{8}$$

In the light of the above discussion, and taking into account the ranges specified in (8) above, we arrive at the following figures providing the two-dimensional and three-dimensional regions of the reachable workspace:

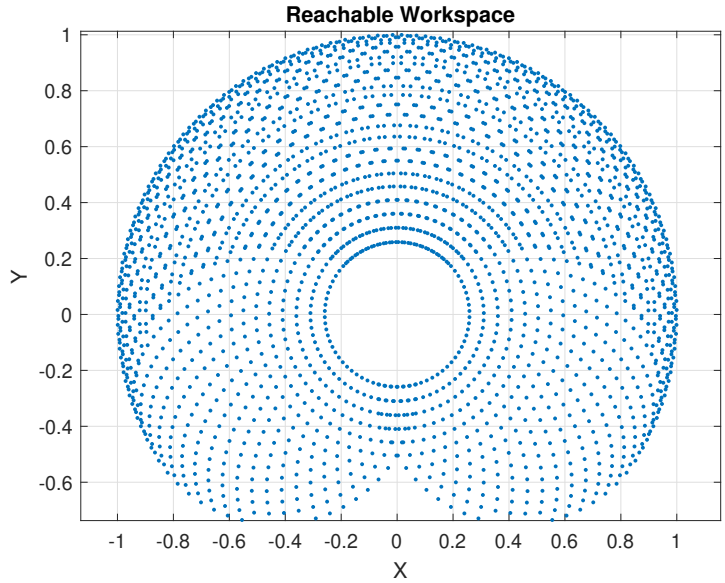

Figure 2: Reachable Workspace in 2D.

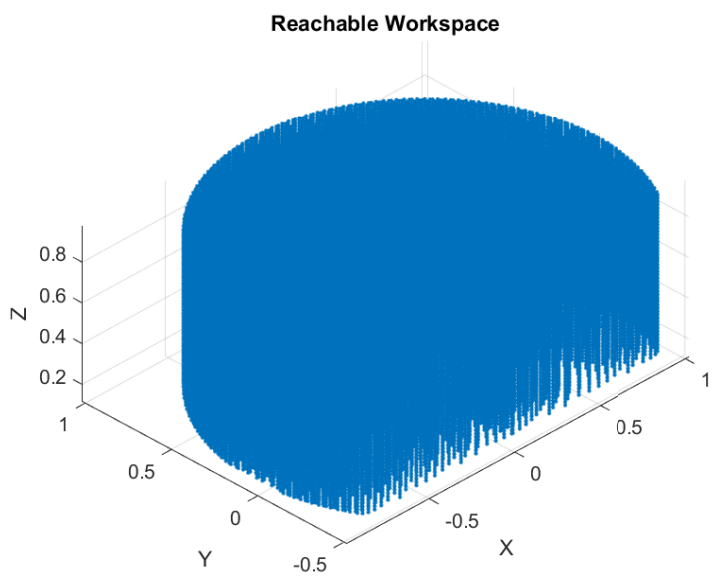

Figure 3: Reachable Workspace in 3D.

3 Manipulability Analysis

This chapter addresses the manipulability of the supporting structure of a SCARA robot. Recall that, the *manipulability* of a manipulator is a measure of the capacity to change the end effector's position

as a function of the joint configuration. In particular, the larger is the manipulability of a robot, the greater is range of possible motions at that configuration. The manipulability measure reduces to zero when the robot is in a singular configuration and cannot develop forces and velocities in at least one direction.

In the case at hand, to verify the end-effector's ability to generate velocities and forces along a given direction, once a manipulator configuration has been assigned, we can use the *velocity manipulability ellipsoid* or the *force manipulability ellipsoid*, defined via the relations

$$\boldsymbol{v}_e^T \left(\boldsymbol{J}(\boldsymbol{q}) \boldsymbol{J}(\boldsymbol{q})^T \right)^{-1} \boldsymbol{v}_e = 1, \tag{9}$$

and

$$\boldsymbol{\gamma}_e^T \left(\boldsymbol{J}(\boldsymbol{q}) \boldsymbol{J}(\boldsymbol{q})^T \right) \boldsymbol{\gamma}_e = 1, \tag{10}$$

respectively. Note that, in (9) and (10) above, $\boldsymbol{J}(\boldsymbol{q})$ denotes the geometric Jacobian, while v_e and γ_e are the vector of end-effector velocities and end-effector forces respectively.

It is worth remarking that the two cores of the quadratic forms in 9 and 10 are in inverse proportionality, hence allowing us to highlight the property of *kineto-statics duality*. This actually entails that the directions of the principal axes of the velocity manipulability ellipsoid coincide with the directions of the principal axes of the force manipulability ellipsoid, while their dimensions are in inverse proportion. For the type of robot under consideration, the supporting structure can be likened to a simple planar manipulator with two arms; the z-axes are all parallel to each other, meaning that the ellipsoids will be defined in the x-y plane. Therefore, the following reduced Jacobian is to be considered

$$\boldsymbol{J}_A(\boldsymbol{q}) = \begin{bmatrix} -a_1 \sin(\theta_1) - a_2 \sin(\theta_1 + \theta_2) & -a_2 \sin(\theta_1 + \theta_2) \\ a_1 \cos(\theta_1) + a_2 \cos(\theta_1 + \theta_2) & a_2 \cos(\theta_1 + \theta_2) \end{bmatrix}. \tag{11}$$

We have selected four possible configurations for the angles (θ_1, θ_2):

$$\begin{cases} (\theta_1, \theta_2) = \left\{ \frac{\pi}{4}, -\frac{\pi}{2} \right\} & \text{well-conditioned ellipse (Fig. 4);} \\ (\theta_1, \theta_2) = \left\{ \frac{\pi}{3}, \frac{\pi}{3} \right\} & \text{medium-conditioned ellipse (Fig. 5);} \\ (\theta_1, \theta_2) = \left\{ \frac{\pi}{3}, \frac{\pi}{4} \right\} & \text{ill-conditioned ellipse (Fig. 6);} \\ (\theta_1, \theta_2) = \left\{ \frac{\pi}{3}, -\frac{\pi}{6} \right\} & \text{almost-singularity (Fig. 7).} \end{cases} \tag{12}$$

Below we show the manipulability ellipse for the four possible chosen configurations.

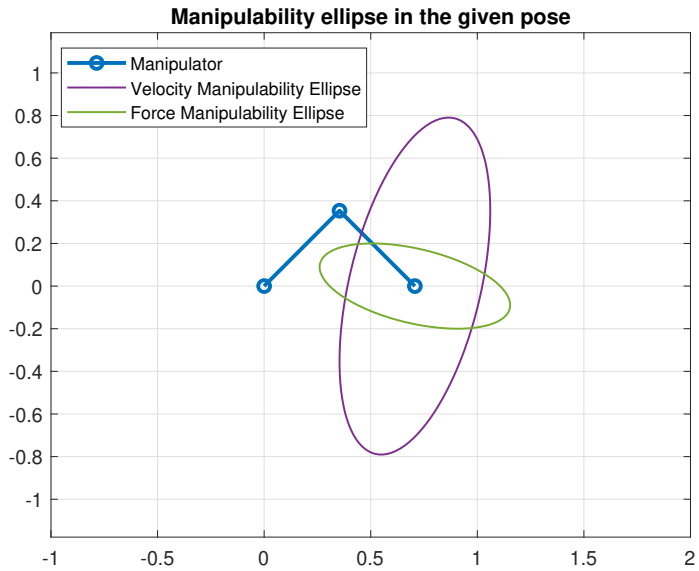

Figure 4: Manipulability ellipse in the first configuration.

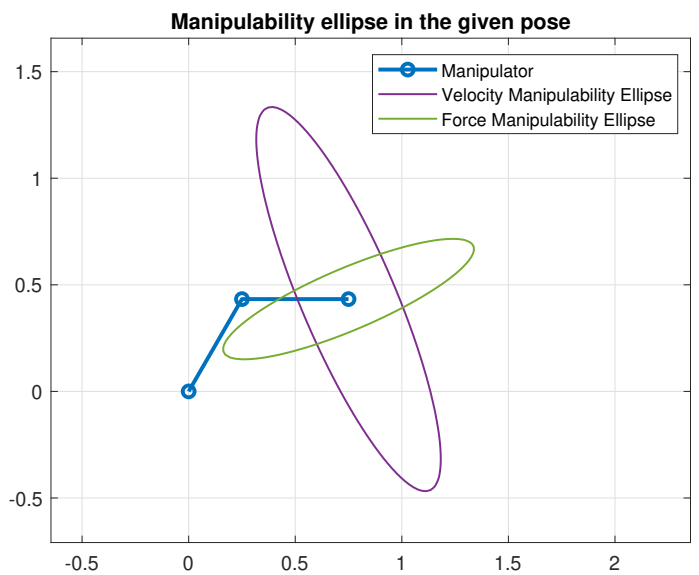

Figure 5: Manipulability ellipse in the second configuration.

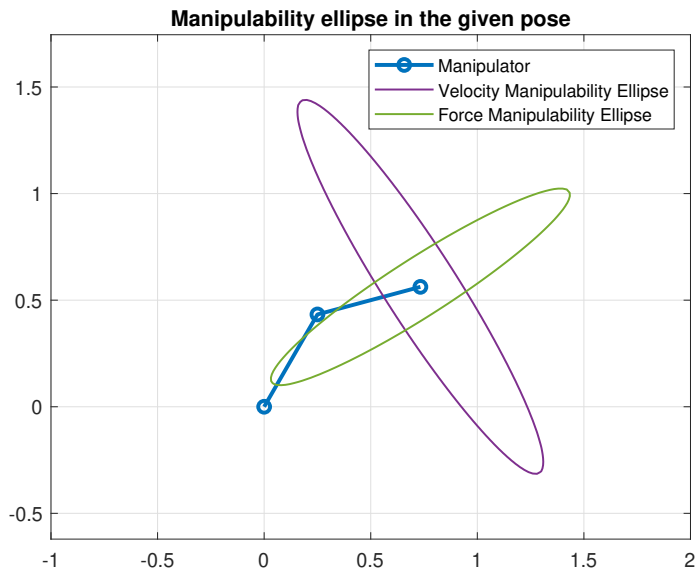

Figure 6: Manipulability ellipse in the third configuration.

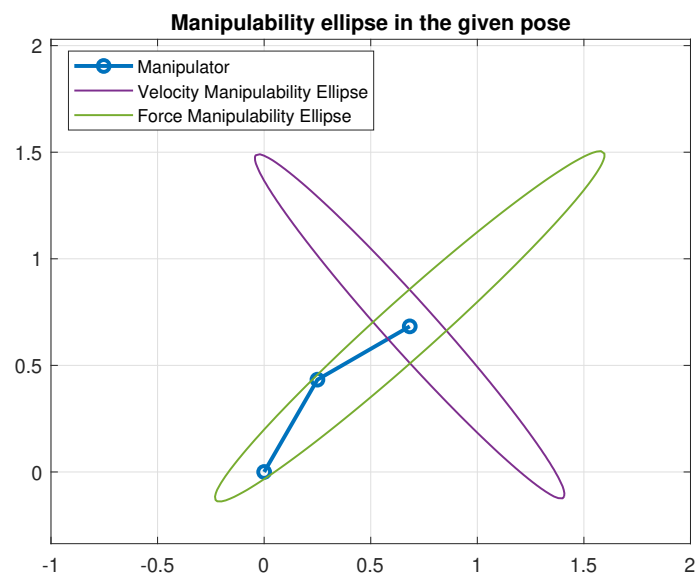

Figure 7: Manipulability ellipse in the fourth configuration.

To conclude this section, let us notice that a global measure of manipulation ability can be expressed in terms of the volume of the ellipsoid. In particular, in the case of a non-redundant manipulator, such volume is proportional to the quantity

$$w(\boldsymbol{q}) = |\det(\boldsymbol{J}(\boldsymbol{q}))|, \tag{13}$$

i.e., the *manipulability measure*, where, as usual, $\boldsymbol{J}$ denotes the geometric Jacobian. Then, in correspondence of each of the configurations fixed above, we find the following possible values of $w(\boldsymbol{q})$:

$$\begin{cases} (\theta_1, \theta_2) = \left\{\frac{\pi}{4}, -\frac{\pi}{2}\right\} & \Longrightarrow \quad w(\boldsymbol{q}) = 0.250; \\ (\theta_1, \theta_2) = \left\{\frac{\pi}{3}, \frac{\pi}{3}\right\} & \Longrightarrow \quad w(\boldsymbol{q}) = 0.216; \\ (\theta_1, \theta_2) = \left\{\frac{\pi}{3}, \frac{\pi}{4}\right\} & \Longrightarrow \quad w(\boldsymbol{q}) = 0.176; \\ (\theta_1, \theta_2) = \left\{\frac{\pi}{3}, -\frac{\pi}{6}\right\} & \Longrightarrow \quad w(\boldsymbol{q}) = 0.125. \end{cases} \tag{14}$$

As the reader may observe, approaching the singularity, the value of $w(\boldsymbol{q})$ decreases, eventually arriving at $w(\boldsymbol{q}) = 0$, at the singular point of the configuration.

4 Trajectory Planning

In this section, we focus on the trajectory planning for the SCARA manipulator. The main aim is to generate a path consisting of a fixed number of points, in such a way to incorporate both straight and circular segments, using a limited number of *via points*, namely, specific points in the operational space, that a manipulator should pass near, rather than exactly through, to ensure smooth and continuous movement. These points are introduced to address issues that arise due to discontinuities in the first derivative of the manipulator's trajectory. In what follows, we discuss in two separate subsections the selection of the path points and the generation of the associated trajectory.

4.1 Selection of Points

Here, we list the twenty points chosen for the trajectory (see Table 2). Note that, P_3, P_{12} and P_{15} are selected as *via points*. Points P_1-P_{20} are selected in such a way to assure the manipulator to perform a specific task. More precisely, our goal is to describe a manipulator executing a pick-and-place task, and then returning to its initial position.

Point	x (m)	y (m)	z (m)	φ (rad)
P_1	-0.60	-0.20	0.30	0
P_2	-0.60	-0.20	0.40	0
P_3	-0.60	-0.20	0.50	$\pi/2$
P_4	-0.40	0.10	0.50	$\pi/2$
P_5	-0.40	0.10	0.50	$\pi/2$
P_6	-0.40	0.10	0.25	0
P_7	-0.40	0.10	0.40	0
P_8	-0.40	0.10	0.70	0
P_9	-0.60	0.10	0.70	0
P_{10}	-0.60	0.30	0.60	0
P_{11}	-0.60	0.30	0.30	0
P_{12}	-0.60	0.30	0.20	0
P_{13}	-0.40	0.50	0.20	$\pi/4$
P_{14}	-0.40	0.50	0.40	$\pi/4$
P_{15}	-0.70	0.40	0.40	0
P_{16}	-0.80	0.20	0.30	0
P_{17}	-0.80	0.10	0.30	0
P_{18}	-0.70	0.05	0.30	0
P_{19}	-0.65	0.06	0.30	0
P_{20}	-0.60	-0.20	0.30	0

Table 2: List of selected points.

For the sake of completeness, we also detail how these points, chosen for trajectory generation, are arranged within the 2D reachable workspace (see Fig. 8).

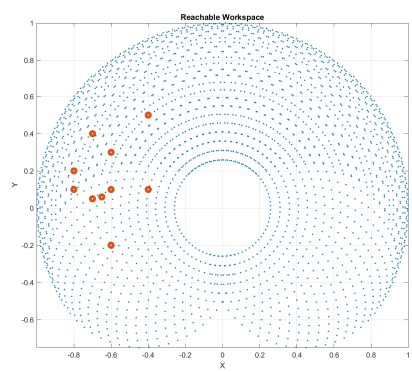

Figure 8: Points in the reachable workspace.

4.2 Trajectory generation

Here we explicitly show the trajectory associated with the selected points. As is apparent from Fig. 9, the generated path consists of sixteen straight sections and two circular sections, with three via points.

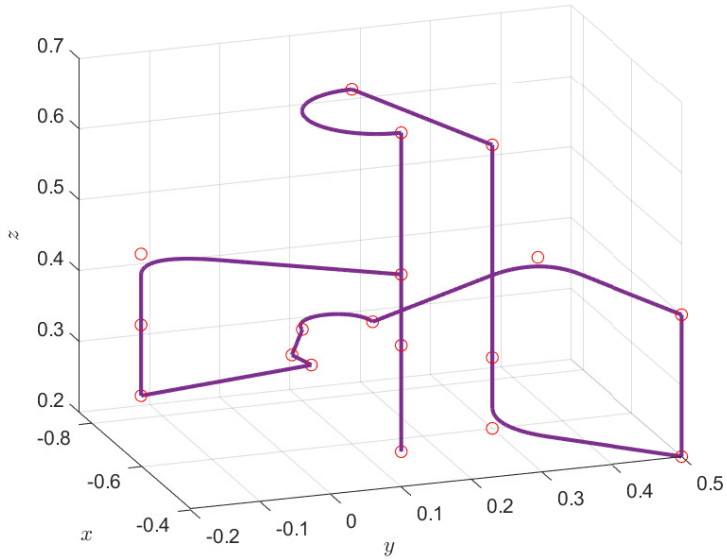

Figure 9: Path in the 3D space

Each segment of the path has been constructed through an assigned parametric representation, using as arc length the function $s(t)$, obtained by means of a *spline* interpolation method.

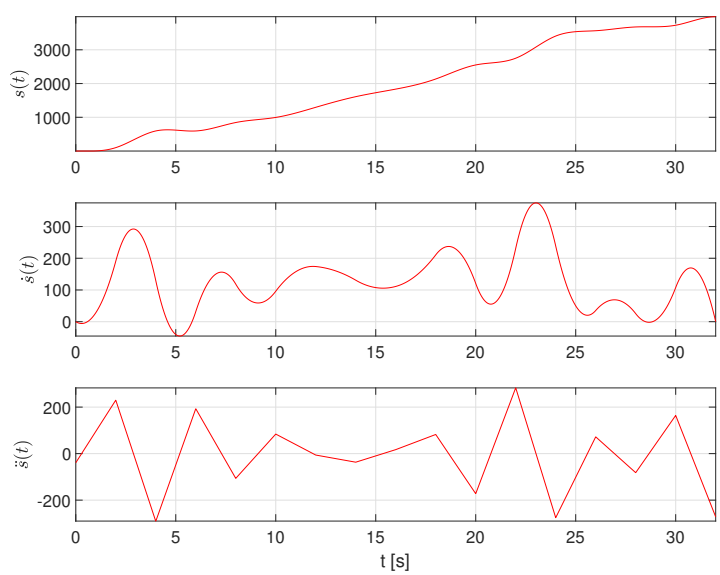

Figure 10: Position, velocity and acceleration functions.

In Fig. 11, 12 and 13, the time evolution of the four components of the end-effector pose $\boldsymbol{x}_e$, velocity $\dot{\boldsymbol{x}}_e$ and acceleration $\ddot{\boldsymbol{x}}_e$ has been reported respectively.

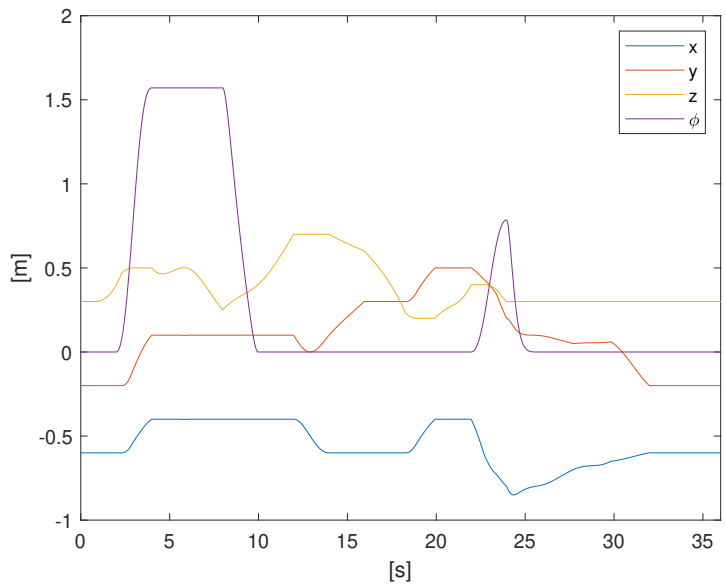

Figure 11: $\boldsymbol{x}_e$ as a function of time.

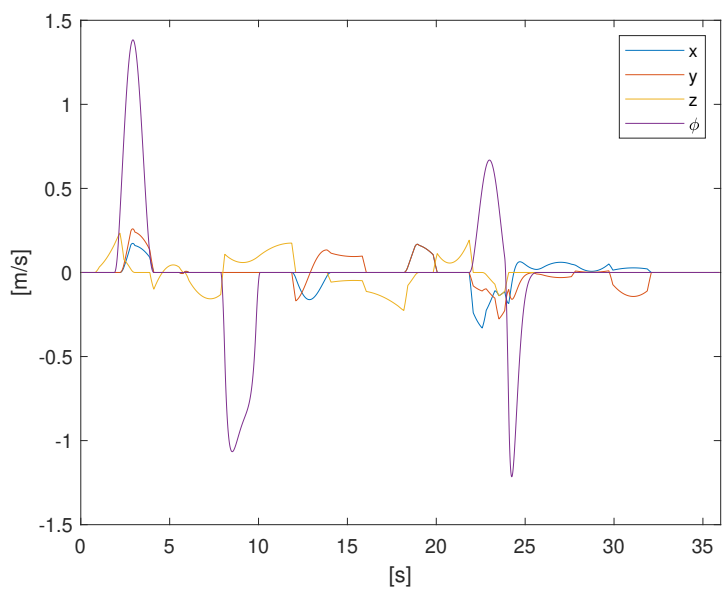

Figure 12: Velocity $\dot{\boldsymbol{x}}_e$ as a function of time.

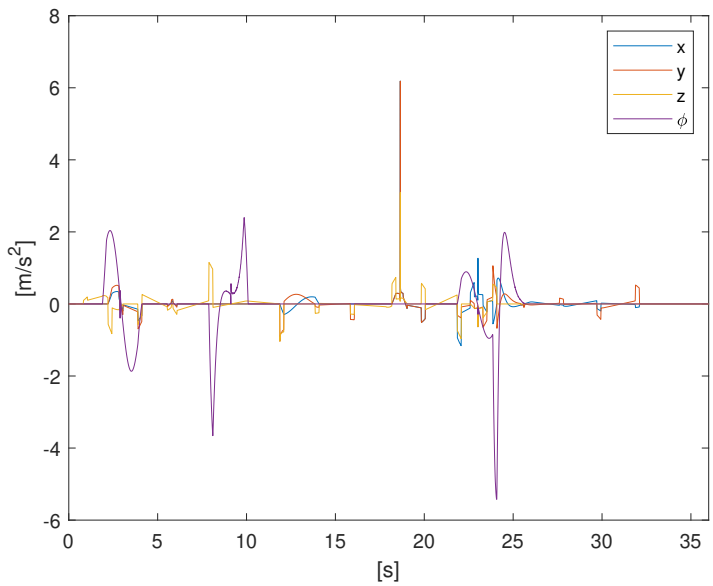

Figure 13: Acceleration $\ddot{\boldsymbol{x}}_e$ as a function of time.

5 Inverse Kinematics Algorithms

Given the end-effector pose, velocity, and acceleration in the operational space, we now aim to determine the corresponding joint positions, velocities, and accelerations in the joint space by means of *Closed Loop Inverse Kinematics* (CLIK, for brevity) algorithms. Such algorithms are based on the definition of a *tracking error* in the operational space, defined as

$$\boldsymbol{e} = \boldsymbol{x}_d - \boldsymbol{x}_e, \tag{15}$$

where $\boldsymbol{x}_d$ and $\boldsymbol{x}_e$ denote the vectors of the desired end-effector pose computed end-effector pose respectively.

The following CLIK algorithms will be implemented for the SCARA manipulator:

- A computationally expensive — yet effective — algorithm that performs the inverse of the analytical Jacobian.
- A less computationally demanding algorithm, albeit with slightly degraded error tracking capabilities, based on the computation of the analytical Jacobian transpose.
- For a redundant manipulator, a modified version of the first CLIK algorithm, which requires the computation of the Jacobian right pseudo-inverse and uses the redundant DOFs to maximize a secondary objective function.
- A second-order CLIK algorithm that determines the corresponding joint accelerations, necessary for specifying the vector of desired joint accelerations in control schemes.

If we now differentiate equation (15) with respect to time, we obtain

$$\dot{\boldsymbol{e}} = \dot{\boldsymbol{x}}_d - \dot{\boldsymbol{x}}_e, \tag{16}$$

which, recalling the differential kinematics equation

$$\dot{\boldsymbol{x}}_e = \boldsymbol{J}_A(\boldsymbol{q})\dot{\boldsymbol{q}}, \tag{17}$$

can be written as

$$\dot{\boldsymbol{e}} = \dot{\boldsymbol{x}}_d - \boldsymbol{J}_A(\boldsymbol{q})\dot{\boldsymbol{q}}. \tag{18}$$

The idea behind the algorithms presented below is to express the vector of joint velocities as a function of the tracking error, in such a way as to obtain a differential equation in the error, whose solution determines its dynamics.

5.1 Jacobian Inverse

Assuming that the matrix J_A is square and nonsingular, then a suitable choice for $\dot{q}$ is given by

$$\dot{\boldsymbol{q}} = \boldsymbol{J}_A^{-1}(\boldsymbol{q})(\dot{\boldsymbol{x}}_d + K\boldsymbol{e}), \tag{19}$$

where $\boldsymbol{K}$ is positive-definite matrix. If we substitute (19) in (18), we obtain the linear differential equation in the error

$$\dot{\boldsymbol{e}} + \boldsymbol{K}\boldsymbol{e} = \boldsymbol{0}. \tag{20}$$

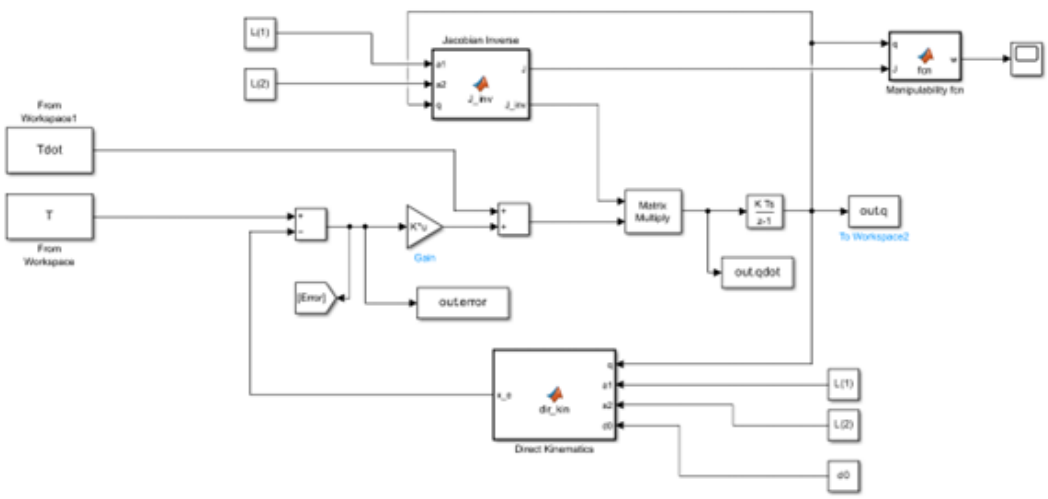

Figure 14: Jacobian Inverse Simulink scheme.

The global asymptotic stability of the system is guaranteed by the positive-definiteness of matrix $\boldsymbol{K}$. Moreover, the error tends to $\boldsymbol{0}$ along the system trajectories with a convergence rate that depends on the eigenvalues of matrix $\boldsymbol{K}$. Consequently, the matrix $\boldsymbol{K}$ was chosen as

$$\boldsymbol{K} = \mathrm{diag}[1000, 1000, 1000, 600]. \tag{21}$$

Running the Simulink model in Fig 8 over the time interval $[0, 36]s$ yields the following results.

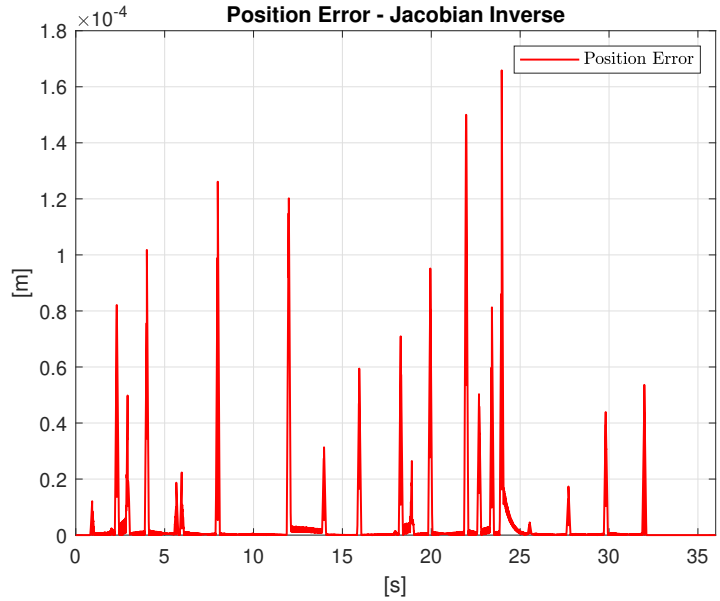

Figure 15: Norm of the end-effector position error.

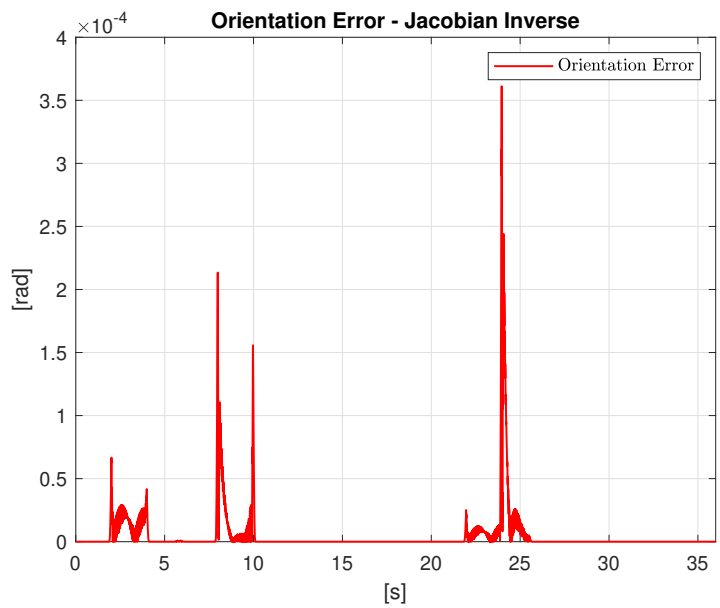

Figure 16: Norm of the end-effector orientation error.

The position and orientation errors, as shown in Fig.15 and Fig.16, are both maintained within an *accuracy* range of 1 mm.

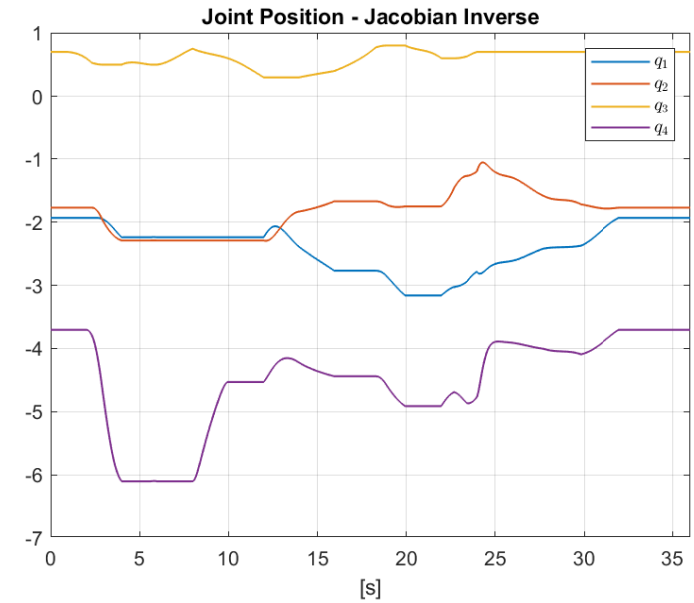

Figure 17: Joint positions.

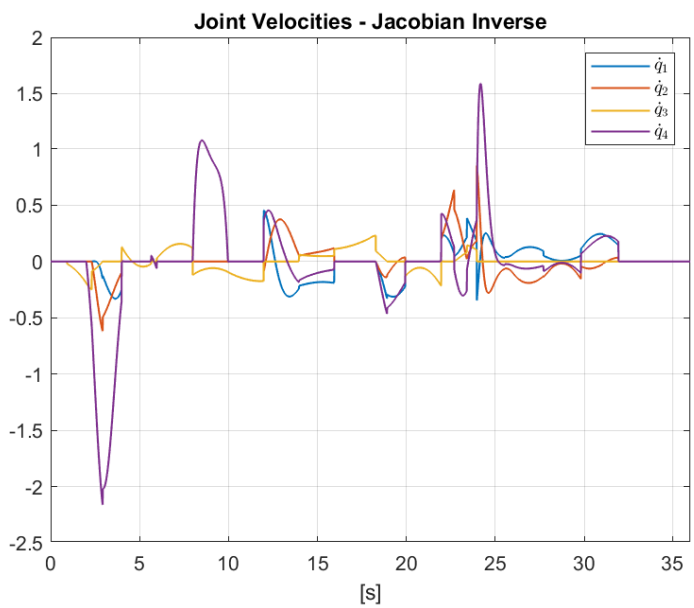

Figure 18: Joint velocities.

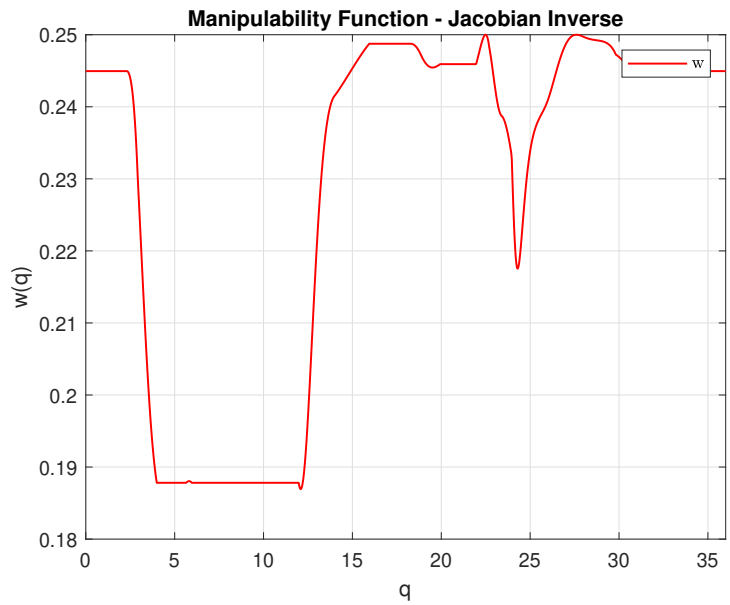

Figure 19: Manipulability measure.

The graph in Fig. 19 shows the evolution of the manipulability measure defined in Equation (13). During the time intervals $[4, 6]s$ and $[24, 26]s$, the value of the joint velocities increases (c.f. Fig. 18) as the manipulability measure decreases. This correlation shows that the manipulator is approaching a singular configuration.

5.2 Jacobian Transpose

A suitable alternative to the previous algorithm is represented by the choice

$$\dot{\boldsymbol{q}} = \boldsymbol{J}_A^T(\boldsymbol{q}) K \boldsymbol{e}. \tag{22}$$

Given that $\boldsymbol{K}$ is a positive-definite matrix and that the transpose of the analytical Jacobian appears in lieu of its invers, this selection for $\dot{\boldsymbol{q}}$ leads to a computationally simpler algorithm and avoids issues related to kinematic and representation singularities, where the Jacobian $\boldsymbol{J}_A$ might be singular or near-singular, making $\boldsymbol{J}_A^{-1}$ either undefined or unstable. The only drawback of this approach is that it may not ensure perfect tracking of the error when the reference input $\boldsymbol{x}_d$ varies rapidly in time.

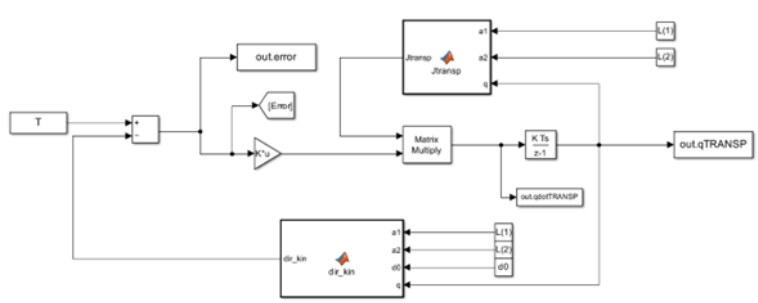

Figure 20: Jacobian transpose Simulink scheme.

The following choice for matrix $\boldsymbol{K}$ has been made:

$$\boldsymbol{K} = \text{diag}[700, 900, 900, 400] \tag{23}$$

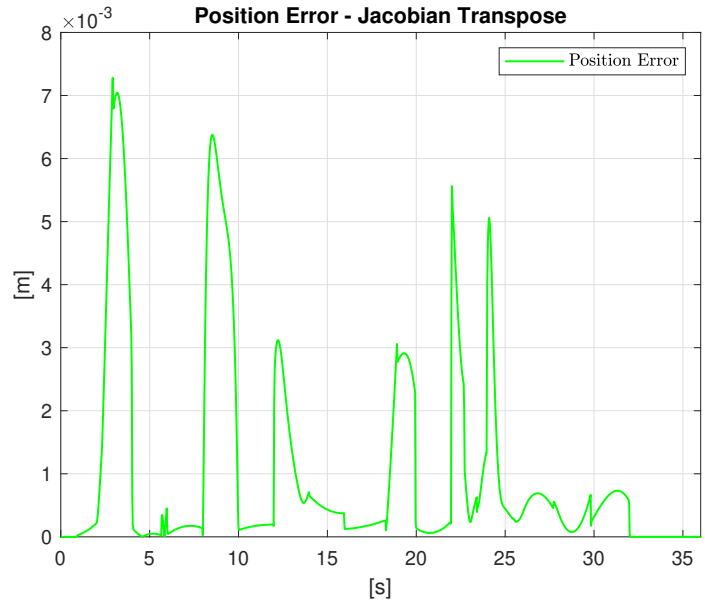

Figure 21: Norm of the end-effector position error.

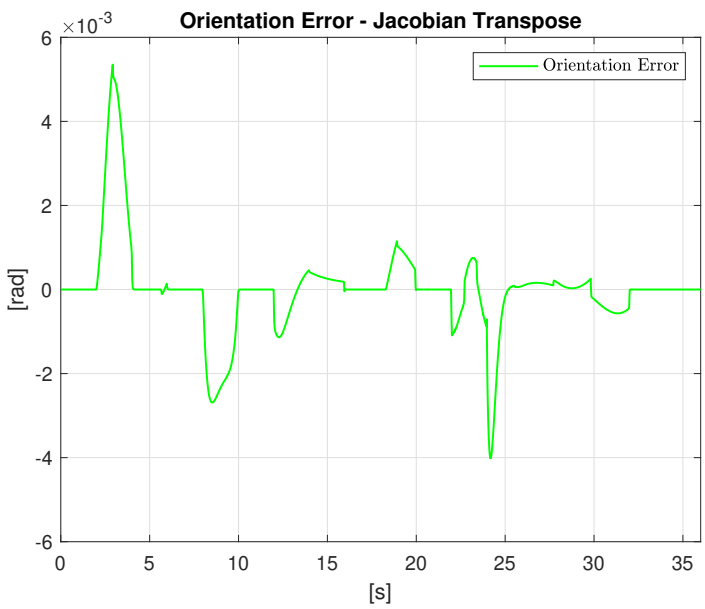

Figure 22: Norm of the end-effector orientation error.

Comparing the graphs from Fig. 23 and Fig. 22 with those from Fig. 15 and Fig. 16, it is clear that

the performance of this second scheme is slightly degraded compared to the first one. This is evident from the position and orientation errors, which are maintained within an accuracy range that is an order of magnitude greater.

5.3 Jacobian Pseudo-Inverse

In a non-redundant system, we can introduce functional redundancy by relaxing a component in the operational space. This allows the robot to perform additional tasks or optimize certain criteria. Moreover, constraints can be imposed on the structure through internal motions to maximize an objective function. Under these conditions, the expression for joint velocity can be refined as follows

$$\dot{\boldsymbol{q}} = \boldsymbol{J}_A^{\dagger}(\boldsymbol{q})(\dot{\boldsymbol{x}}_e + K\boldsymbol{e}) + \big(I_n - \boldsymbol{J}_A^{\dagger}(\boldsymbol{q})\boldsymbol{J}_A(\boldsymbol{q})\big)\dot{\boldsymbol{q}}_0, \tag{24}$$

where

- $\boldsymbol{J}_A^{\dagger}(\boldsymbol{q})$ is the right pseudo-inverse of the Jacobian matrix;
- $\boldsymbol{K}$ is a positive-definite matrix;
- $\boldsymbol{I}_n$ is an $n \times n$ identity matrix;
- $\dot{\boldsymbol{q}}_0$ is an arbitrary joint velocity vector that can be used to satisfy additional criteria or constraints and is typically chosen as

$$\dot{\boldsymbol{q}}_0 = k_0 \left(\frac{\partial w}{\partial \boldsymbol{q}} \right), \tag{25}$$

  where $k_0 > 0$ is a scalar, and w is a secondary objective function to maximize, compatible to the primary objective, i.e., the kinematic constraint given by $\boldsymbol{v}_e = \boldsymbol{J}(\boldsymbol{q})\dot{\boldsymbol{q}}$. In this project, we choose to maximize the manipulability measure already defined in (13) for a non redundant manipulator. Given that we have made the SCARA manipulator functionally redundant, the manipulability measure is defined as

$$w(\boldsymbol{q}) = \sqrt{\det(\boldsymbol{J}\boldsymbol{J}^T)}. \tag{26}$$

The operational space component to be relaxed was selected as the x-component. Hence the Jacobian of the structure is obtained by eliminating the first row from the matrix in equation (7)

$$\boldsymbol{J} = \begin{bmatrix} a_1\cos(\theta_1) + a_2\cos(\theta_1+\theta_2) & a_2\cos(\theta_1+\theta_2) & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 1 & 1 & 0 & -1 \end{bmatrix} \tag{27}$$

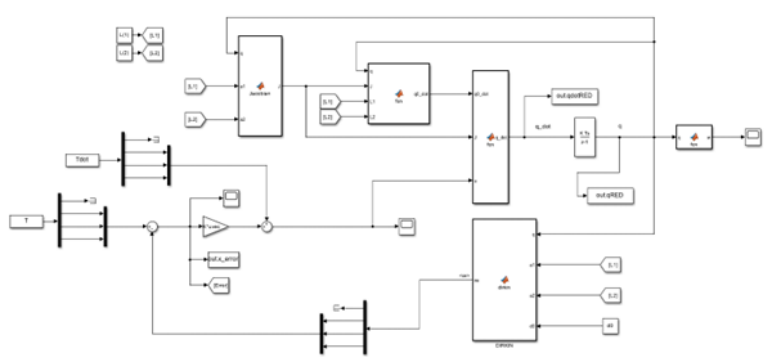

Figure 23: Jacobian Pseudo-Inverse Simulink Scheme.

Finally, the values for matrix $\boldsymbol{K}$ and the scalar k_0 are specified:

$$K = \text{diag}[800, 800, 480], \quad k_0 = 10, \tag{28}$$

leading to the following results.

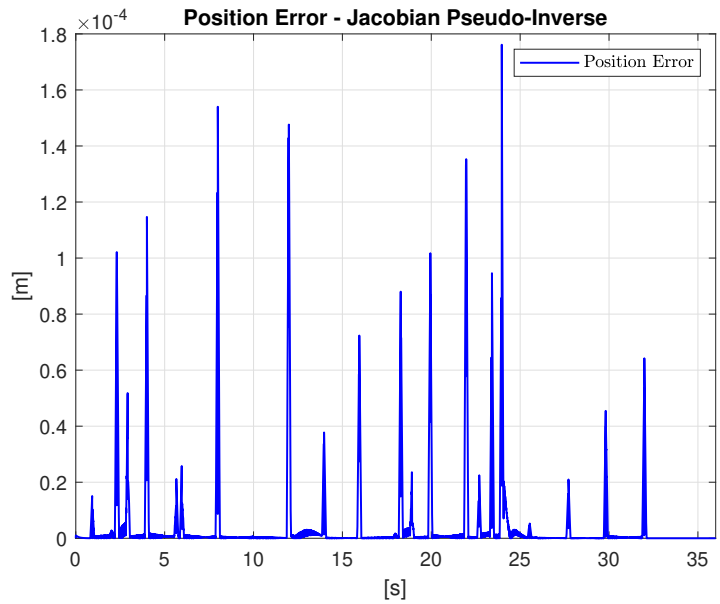

Figure 24: Norm of the end-effector position error.

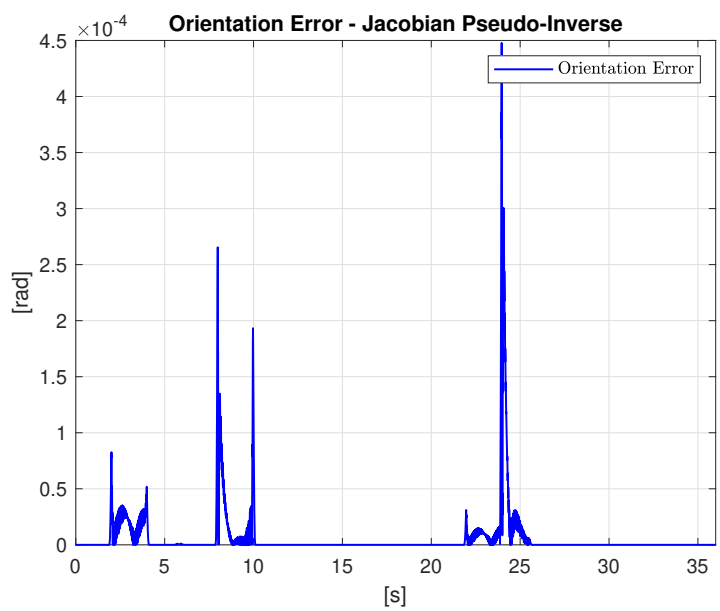

Figure 25: Norm of the end-effector orientation error.

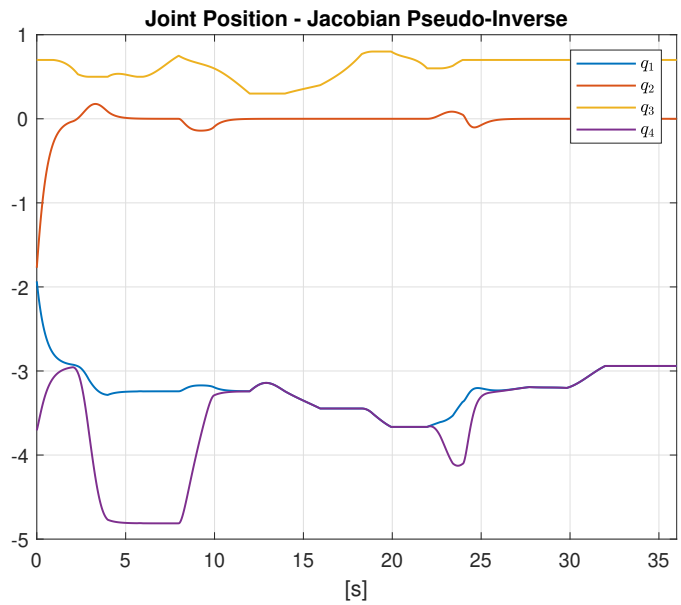

Figure 26: Joint positions.

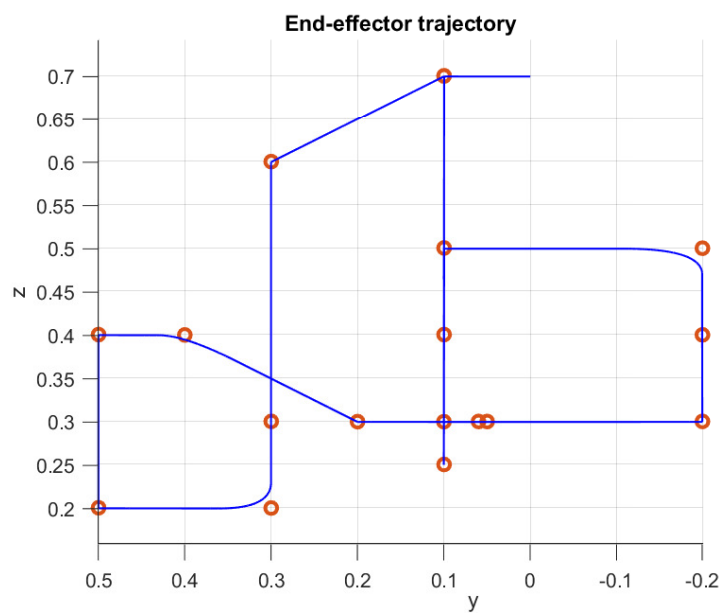

Figure 27: Trajectory on $y - z$ plane.

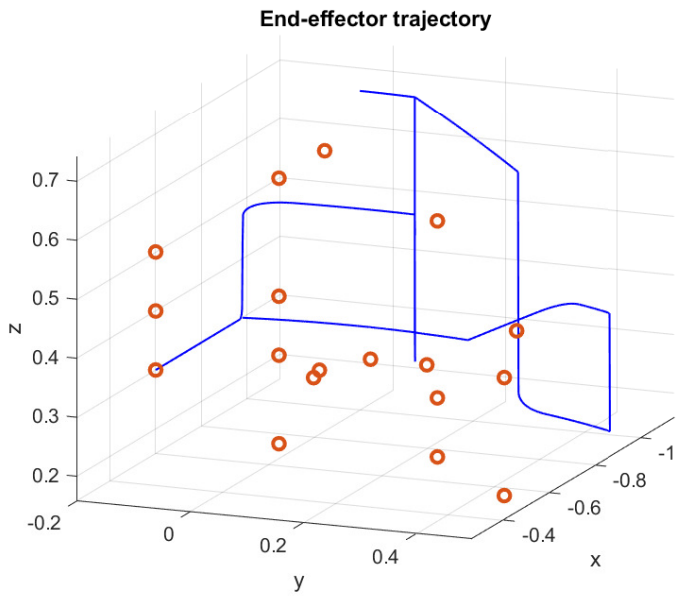

Figure 28: Trajectory in 3D space.

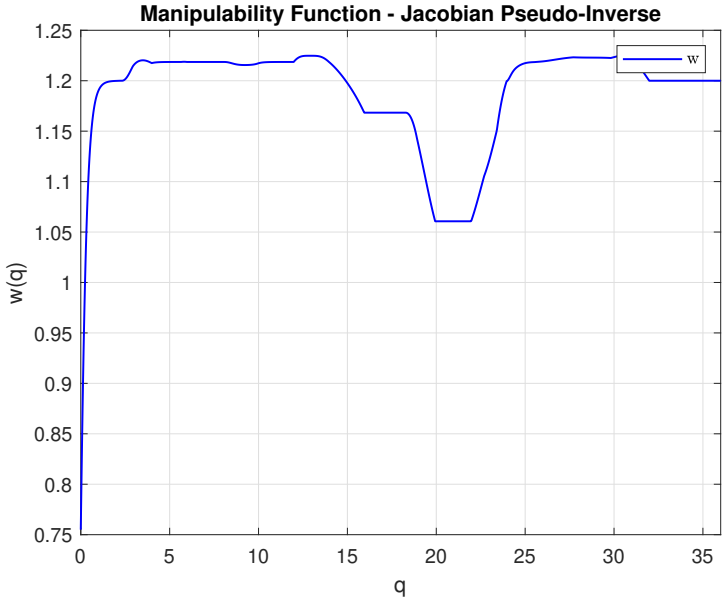

Figure 29: Manipulability Measure.

The position and orientation errors — as shown in Fig. 24 and in Fig. 25 — remain unchanged compared to those from the first CLIK scheme. This is becaise the vector of joint velocities $\dot{\boldsymbol{q}}_0$ induces internal motions that do not affect the end-effector position and orientation. When comparing Fig. 19 and Fig. 29, it is evident that the manipulability measurement improves. The use of the pseudoinverse of the Jacobian in a redundant system enhances manipulability by distributing errors, minimizing joint

velocities, increasing numerical stability, and avoiding singularities. This results in greater flexibility and robustness in the robot's movements. In Fig. 27, the trajectory on the $y - z$ plane shows no noticeable difference from the original path. However, significant discrepancies are observed on the $x - y$ and $x - z$ planes. These differences arise because the $x-$component is influenced by internal motion, as demonstrated by comparing Fig.28 with Fig. 9.

5.4 Second-order algorithm

The inverse kinematics algorithms discussed previously are classified as first-order algorithms. These algorithms convert a motion trajectory, characterized by the position and orientation of the end-effector, into corresponding joint positions and velocities. However, control applications often require the inversion of a motion trajectory that includes not only position and velocity but also acceleration parameters. Consequently, a second-order CLIK algorithm is to be implemented.

To address this need, the joint accelerations vector can be represented as follows:

$$\ddot{\boldsymbol{q}} = \boldsymbol{J}_A^{-1}(\boldsymbol{q}) \left(\ddot{\boldsymbol{x}}_d + \boldsymbol{K}_D \dot{\boldsymbol{e}} + \boldsymbol{K}_P \boldsymbol{e} - \dot{\boldsymbol{J}}_A(\boldsymbol{q}) \dot{\boldsymbol{q}} \right), \tag{29}$$

where $\boldsymbol{K}_P$ and $\boldsymbol{K}_D$ are positive-definite matrices.

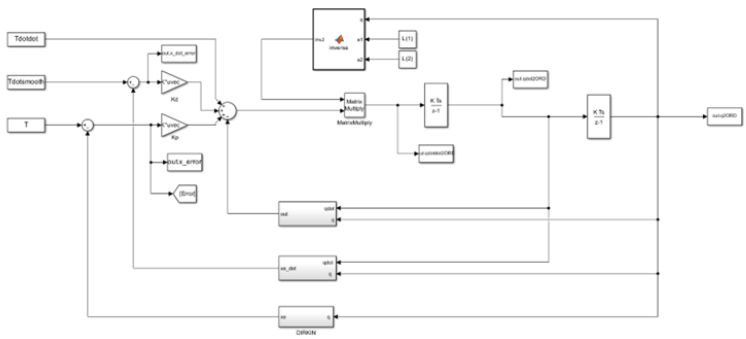

Figure 30: Second-order algorithm Simulink scheme.

The matrices $\boldsymbol{K}_P$ and $\boldsymbol{K}_D$ chosen for the simulation are:

$$\begin{aligned} \boldsymbol{K}_D &= \mathrm{diag}[100, 100, 100, 100], \\ \boldsymbol{K}_P &= \mathrm{diag}[500, 500, 500, 500]. \end{aligned} \tag{30}$$

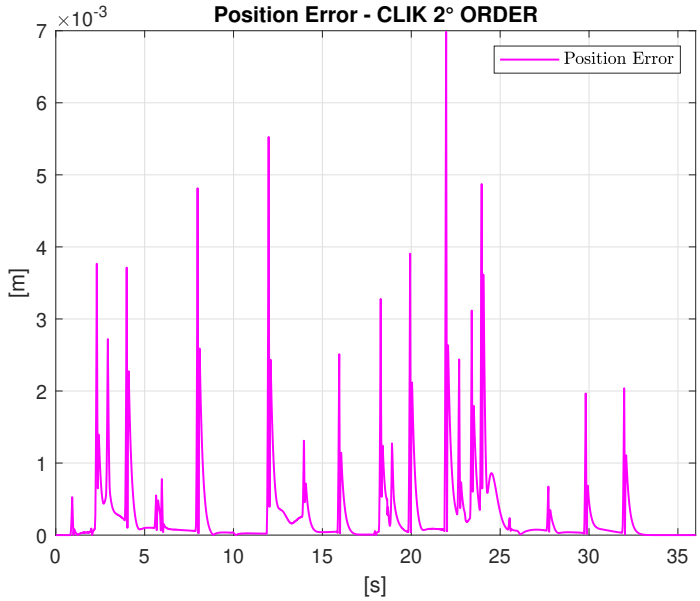

Figure 31: Norm of the end-effector position error

Figure 32: Norm of the end-effector orientation error

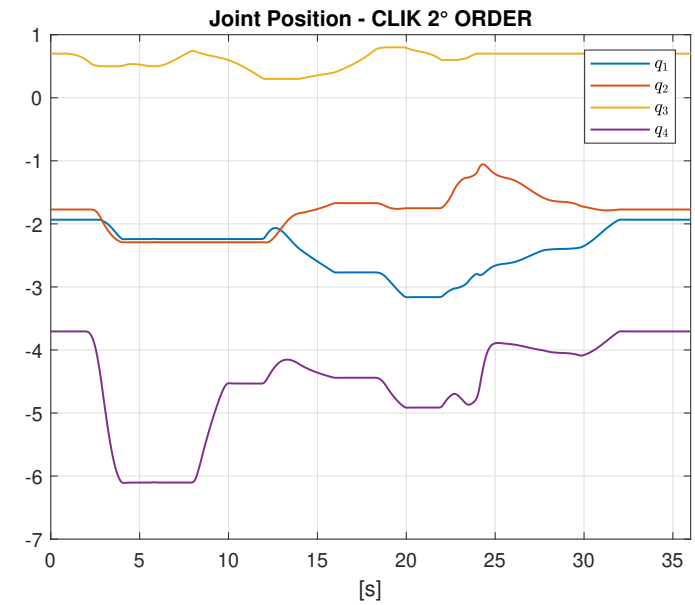

Figure 33: Joint positions.

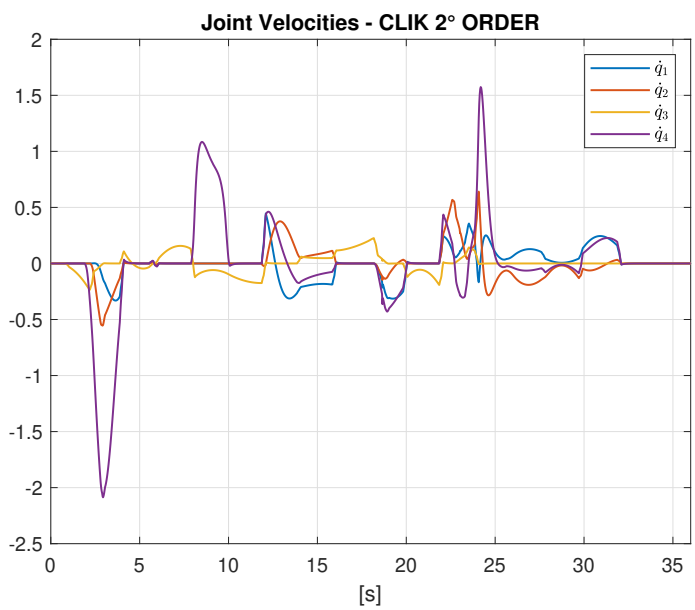

Figure 34: Joint velocities.

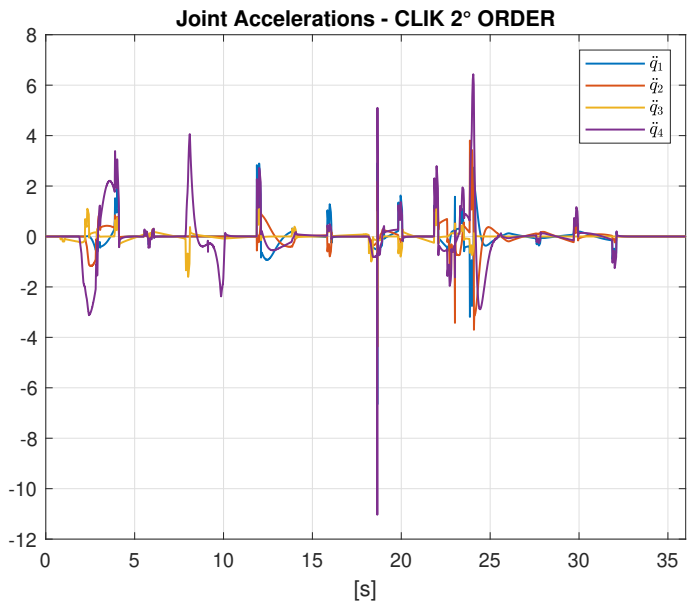

Figure 35: Joint accelerations.

6 Motion Control

In this section, the dynamic model of a SCARA manipulator will be derived using the dynamic model of a robot manipulator obtained with one of the two possible approaches, namely, the so-called Lagrangian formulation.

The dynamic model of a robot manipulator has two fundamental properties: the skew symmetry of the matrix $\dot{\boldsymbol{B}} - 2\boldsymbol{C}$, and the linearity with respect to a suitable set of dynamic parameters. Once this matrix is derived, we will be able to define robust control and adaptive control.

6.1 Dynamic Model

The dynamic model of a generic manipulator can be modeled via a suitable Lagrangian formulation, where the Lagrange equation can be expressed in the following compact form [1]:

$$\frac{\mathrm{d}}{\mathrm{d}t}\left(\frac{\partial \mathcal{L}}{\partial \dot{\boldsymbol{q}}}\right)^T - \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{q}}\right)^T = \boldsymbol{\xi}, \tag{31}$$

where $\mathcal{L}$ is the Lagrangian of the mechanical system — defined as the difference between the total kinetic energy and the potential energy of the system, $\boldsymbol{q}$ is the vector of generalized coordinates, and $\boldsymbol{\xi}$ denotes the vector of generalized forces.

As is well known [1], under suitable assumptions, the joint space dynamic model can be expressed by means of the following relation

$$\boldsymbol{B}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{F}_v\dot{\boldsymbol{q}} + g(\boldsymbol{q}) = \boldsymbol{\tau}, \tag{32}$$

where $\boldsymbol{B}(\boldsymbol{q})$ denotes the *inertia matrix*, $C(\boldsymbol{q}, \dot{\boldsymbol{q}})$ is the matrix containing the Coriolis and the centrifugal effects, $\boldsymbol{F}_v$ is the diagonal matrix of viscous friction coefficients, $\boldsymbol{g}(\boldsymbol{q})$ is the gravity torque vector at the joint axes, and τ is the actuation torque vector. Let us now better detail these matrices. For what concerns the inertia matrix $\boldsymbol{B}(\boldsymbol{q})$, it worth observing that, under the assumption taken in consideration

for the model analyzed in this project — namely, that the angular velocity of each rotor is only due to its own spinning we can express this matrix as

$$\boldsymbol{B}(\boldsymbol{q}) = \boldsymbol{B}_l(\boldsymbol{q}) + \boldsymbol{B}_m, \tag{33}$$

where

$$\begin{aligned}\boldsymbol{B}_l(\boldsymbol{q}) = m_{l_1}\boldsymbol{J}_P^{(l_1)^T} + \boldsymbol{J}_O^{(l_1)^T}\boldsymbol{I}_{l_1}\boldsymbol{J}_O^{(l_1)} + m_{l_2}\boldsymbol{J}_P^{(l_2)^T} + \\ \boldsymbol{J}_O^{(l_2)^T}\boldsymbol{I}_{l_2}\boldsymbol{J}_O^{(l_2)} + m_{l_4}\boldsymbol{J}_P^{(l_4)^T} + \boldsymbol{J}_O^{(l_4)^T}\boldsymbol{I}_{l_4}\boldsymbol{J}_O^{(l_4)},\end{aligned} \tag{34}$$

while

$$\boldsymbol{B}_m = \boldsymbol{K}_r\boldsymbol{I}_m\boldsymbol{K}_r. \tag{35}$$

Note that, in (35) and (35) above we have set

- m_{l_i} for the mass of i-th augmented link;
- $\boldsymbol{I}_{l_i}$ for the inertia moment of i-th augmented link;
- I_{m_i} for the inertia moment of i-th motor;
- $\boldsymbol{J}_P^{(l_i)}$ to denote the Jacobian related to the linear velocity of the i-th augmented link CoM;
- $\boldsymbol{J}_O^{(l_i)}$ for the Jacobian related to the angular velocity of the i-th link CoM;
- $\boldsymbol{J}_O^{(m_i)}$ for the Jacobian related to the angular velocity of the i-th motor CoM.
- $\boldsymbol{K}_r = \mathrm{diag}[k_{r1}, \ldots, k_{rn}]$ is the diagonal matrix of gear reduction ratios.

From this peculiar structure of $\boldsymbol{B}(\boldsymbol{q})$, some noteworthy consequences can be derived. Namely, $\boldsymbol{B}(\boldsymbol{q})$ is symmetric, being the sum of the two symmetric matrices (34) and (35); it is also positive definite since, in our assumptions, it is directly related to the kinetic energy, the letter being non-negative, and 0, iff $\dot{\boldsymbol{q}} = 0$. Next, for what concerns the matrix $\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})$, we observe that its entries actually consist of the Christoffel symbols of the first type, i.e.,

$$\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = (c_{ij}), \quad c_{ij} = \sum_{k=1}^{4} c_{ijk}\dot{q}_k, \quad c_{ijk} = \frac{1}{2}\left(\frac{\partial b_{ij}}{\partial q_k} + \frac{\partial b_{ik}}{\partial q_j} - \frac{\partial b_{ik}}{\partial q_i}\right), \tag{36}$$

where $\boldsymbol{B}(\boldsymbol{q}) = (b_{ij})$ are the components of the inertia matrix. The diagonal matrix F_v of viscous friction coefficients is defined by

$$\boldsymbol{F}_v = \mathrm{diag}[k_{r_1}^2 F_{m_1}, k_{r_2}^2 F_{m_2}, k_{r_3}^2 F_{m_3}, k_{r_4}^2 F_{m_4}], \tag{37}$$

where each component F_{m_i} represents the viscous coefficient relative to the i-th motor, while k_{r_i} is the transmission constant of the i-th motor. Lastly, the gravity torque vector at the joint axes $\boldsymbol{g}(\boldsymbol{q})$ was set, according to the specific model considered, as

$$\boldsymbol{g}(\boldsymbol{q}) = [0, 0, -9.81 m_{l_3}^2, 0]^T. \tag{38}$$

6.2 Robust Control

Robust control is a design strategy in control systems that focuses on ensuring a system maintains desirable performance despite uncertainties and variations in system parameters. Unlike classical control, robust control is designed to perform effectively even when faced with external disturbances, parameter variations, or model inaccuracies. The input law will be written as

$$\boldsymbol{u} = \widehat{\boldsymbol{B}}(\boldsymbol{q})\boldsymbol{y} + \widehat{\boldsymbol{n}}(\boldsymbol{q}, \dot{\boldsymbol{q}}), \tag{39}$$

where $\widehat{\boldsymbol{B}}(\boldsymbol{q})$ represents the estimation of the matrix $\boldsymbol{B}(\boldsymbol{q})$, while $\widehat{\boldsymbol{n}}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \widehat{\boldsymbol{C}}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{F}_v\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q})$ provides the estimation of terms related to joint velocities and potential energy. We will represents the uncertainty $\widetilde{\boldsymbol{B}}(\boldsymbol{q})$ and $\widetilde{\boldsymbol{n}}(\boldsymbol{q}, \dot{\boldsymbol{q}})$ as

$$\widetilde{\boldsymbol{B}}(\boldsymbol{q}) = \widehat{\boldsymbol{B}}(\boldsymbol{q}) - \boldsymbol{B}(\boldsymbol{q}), \qquad \widetilde{\boldsymbol{n}}(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \widehat{\boldsymbol{n}}(\boldsymbol{q}, \dot{\boldsymbol{q}}) - \boldsymbol{n}(\boldsymbol{q}, \dot{\boldsymbol{q}}). \tag{40}$$

The control law $\boldsymbol{y}$ has been created with a feedforward action and a feedback action:

$$\boldsymbol{y} = \ddot{\boldsymbol{q}}_d + \boldsymbol{K}_D\dot{\widetilde{\boldsymbol{q}}} + \boldsymbol{K}_p\widetilde{\boldsymbol{q}} + \boldsymbol{w}. \tag{41}$$

Note that, here $\boldsymbol{K}_D$ and $\boldsymbol{K}_P$ are two gain matrices, which ensure stabilization of the error dynamic system matrix, $\ddot{\boldsymbol{q}}_d$ provides a feedforward term, and $\boldsymbol{w}$ is chosen to guarantee robustness to the effects of uncertainty, with the following condion:

$$\boldsymbol{w} = \frac{\rho}{\|\boldsymbol{z}\|}\boldsymbol{z}, \quad \rho > 0 \tag{42}$$

and

$$\boldsymbol{z} = \boldsymbol{D}^T\boldsymbol{Q}\begin{bmatrix}\widetilde{\boldsymbol{q}} \\ \dot{\widetilde{\boldsymbol{q}}}\end{bmatrix} \tag{43}$$

where we set

- ρ for a positive scalar chosen so that
$$\rho \geq \|\boldsymbol{\eta}\|, \tag{44}$$
where $\boldsymbol{\eta}$ denotes the model uncertainty.
- $\boldsymbol{D}$ for a $8x4$ matrix that multiplies the system input, which has been rewritten in terms of the state variables $[\widetilde{\boldsymbol{q}}, \ddot{\widetilde{\boldsymbol{q}}}]$
- $\boldsymbol{Q}$ for an $8x8$ positive definite matrix.

To avoid the chattering phenomenon, the control law is modified to ensure that the error remains within a bounded norm, with $\epsilon > 0$:

$$\boldsymbol{w} = \begin{cases} \frac{\rho}{\|\boldsymbol{z}\|}\boldsymbol{z}, & \text{for } \|z\| \ \geq \epsilon \\ \frac{\rho}{\epsilon} & \text{for } \|\boldsymbol{z}\| < \varepsilon. \end{cases} \tag{45}$$

In order to show the performances of a robust control scheme, we purposely neglected the terms due to Coriolis and centrifugal forces $\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}}$, and considered only the elements of the main diagonal of the inertia matrix $\boldsymbol{B}$. The values of ρ, $\boldsymbol{K}_P$, $\boldsymbol{K}_D$, $\boldsymbol{Q}$ have been chosen as shown below

$$\begin{cases} \rho = 10 \\ \boldsymbol{K}_P = \text{diag}[800, 800, 800, 800] \\ \boldsymbol{K}_D = \text{diag}[300, 300, 300, 300] \\ \boldsymbol{Q} = \text{diag}[10, 10, 10, 10, 0.1, 0.1, 0.1, 0.1]. \end{cases} \tag{46}$$

With this choice of $\boldsymbol{Q}$, greater emphasis was placed on minimizing position error, ensuring better trajectory tracking, albeit with some delay.

The Simulink scheme of the robust control has been represented below:

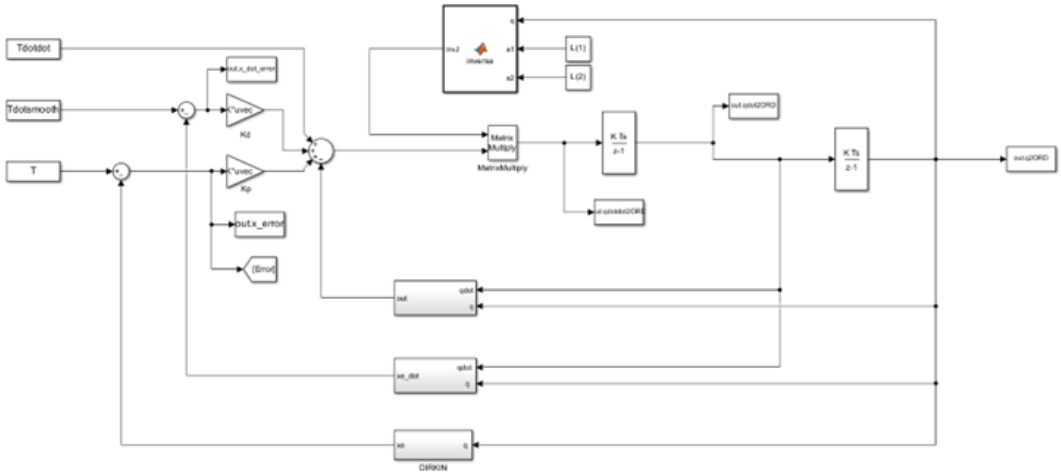

Figure 36: Robust Control Simulink scheme.

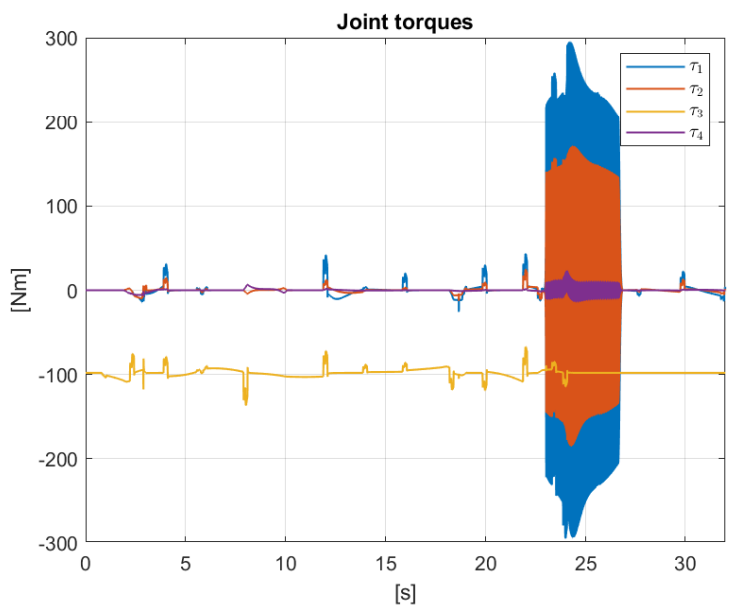

Figure 37: Joint torques with $\varepsilon = 0.003$.

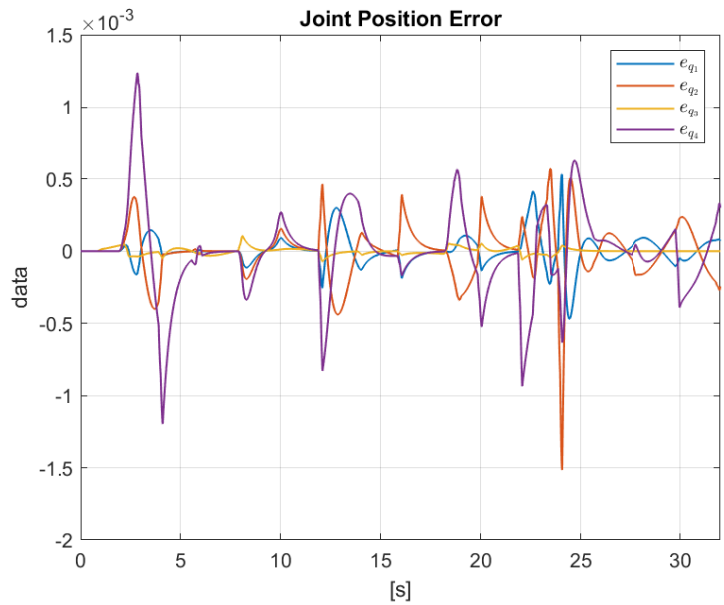

Figure 38: Joint position error with $\varepsilon = 0.01$.

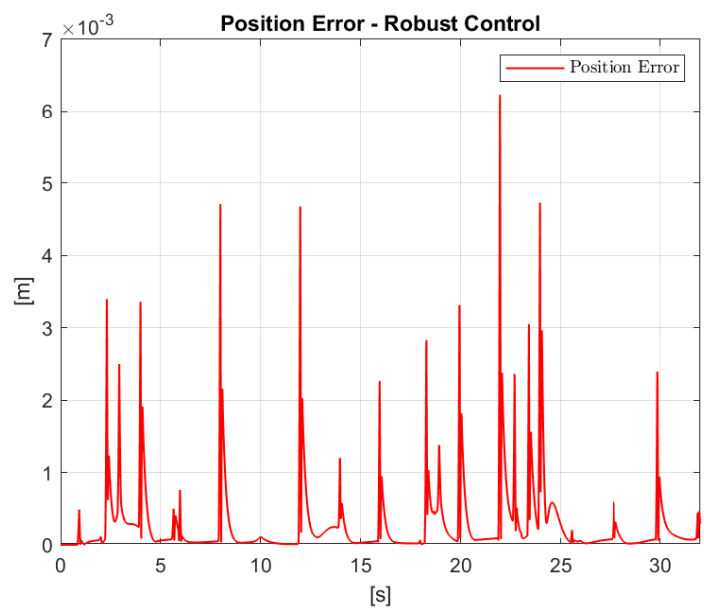

Figure 39: Norm of the end-effector position error with $\varepsilon = 0.01$.

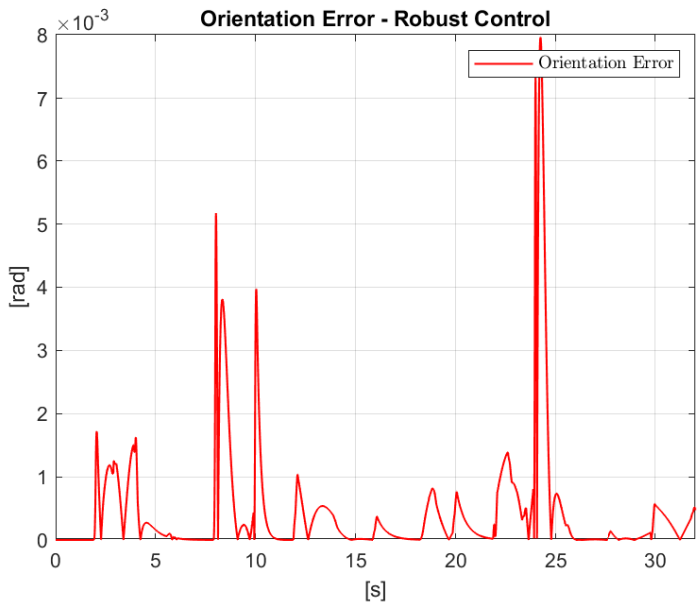

Figure 40: Norm of the end-effector orientation error with $\varepsilon = 0.01$.

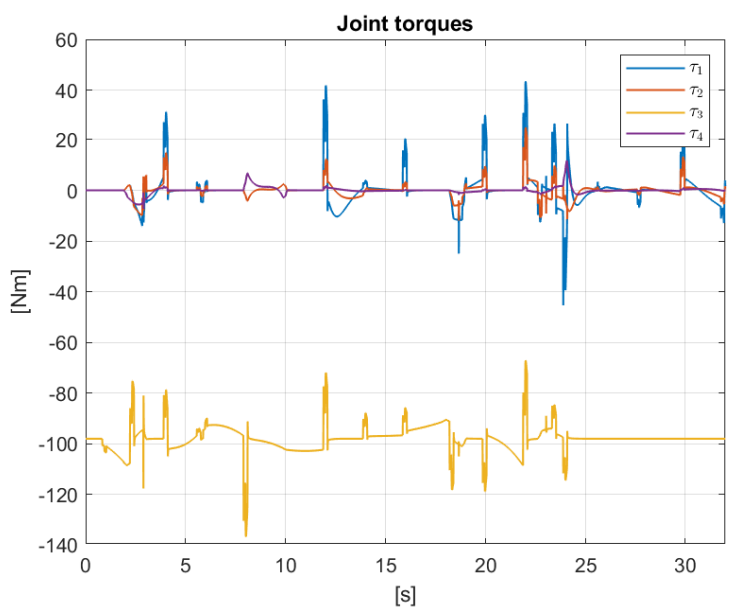

Figure 41: Joint torques with $\varepsilon = 0.01$.

A series of simulations was conducted to determine the appropriate value of ε to avoid the chattering phenomenon. Two values for ε were identified:

- In the first case, $\varepsilon = 0.003$ was chosen, which generated an extremely high value of the joint torques, resulting in the phenomenon of chattering, as shown in Fig. 37.

- In the second case, $\varepsilon = 0.01$ was found to avoid this phenomenon, generating the optimal case, as shown in Fig. 41.

These results indicate that selecting an appropriate ε value is crucial for avoiding chattering and achieving optimal performance.

6.3 Adaptive Control

Adaptive control is a control strategy designed to handle systems where dynamic parameters may vary over time or are initially unknown. It dynamically adjusts the controller parameters to accommodate changes in system conditions or dynamics. The computational model typically has the same structure as that of the true manipulator dynamic model, but parameters estimate could be affected by uncertainty: a solution could be using an adaptive control law based on property of linearity in the parameters of the dynamic model of the manipulator. The dynamic model can be written as:

$$\boldsymbol{B}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{F}_v\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{Y}(\boldsymbol{q}, \dot{q}, \ddot{q})\pi, \tag{47}$$

where $\boldsymbol{\pi}$ is a vector of constant dynamic parameters and $\boldsymbol{Y}$ is the regressor matrix, which depends on joint positions, velocities and accelerations. For the SCARA manipulator, the vector $\boldsymbol{\pi}$ is $(14x1)$:

$$\boldsymbol{\pi} = [m_{l_1}, l_{l_1}, \boldsymbol{I}_{l_1}, \boldsymbol{F}_{m_1}, m_{l_2}, l_{l_2}, \boldsymbol{I}_{l_2}, \boldsymbol{F}_{m_2}, m_{l_3}, l_{l_3}, \boldsymbol{I}_{l_3}, \boldsymbol{F}_{m_3}, m_{l_4}, l_{l_4}, \boldsymbol{I}_{l_4}, \boldsymbol{F}_{m_4}]. \tag{48}$$

Supposing we have parameter estimates instead of true values, the control law must be chosen as:

$$\boldsymbol{u} = \widehat{B}(\boldsymbol{q})\ddot{\boldsymbol{q}}_r + \widehat{\boldsymbol{C}}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}}_r + \widehat{\boldsymbol{F}}\dot{\boldsymbol{q}}_r + \widehat{\boldsymbol{g}}(\boldsymbol{q}) + \boldsymbol{K}_d\sigma = \boldsymbol{Y}(\boldsymbol{q}, \dot{q}, \dot{\boldsymbol{q}}_r, \ddot{\boldsymbol{q}}_r)\widehat{\pi} + \boldsymbol{K}_d\sigma. \tag{49}$$

The quantities appearing in (49) above are defined as follows:

- $\boldsymbol{K}_d$ is a $14x14$ diagonal, positive-definite, whose elements are weighted control actions;
- $\boldsymbol{Y}$ is the $4x14$ regressor matrix;
- σ is the linear combination of position and velocity errors;
- $\dot{\boldsymbol{q}}_r = \dot{\boldsymbol{q}}_d - \Lambda\widetilde{\boldsymbol{q}}$ is the linear combination of position error and desired velocity;
- $\ddot{\boldsymbol{q}}_r = \ddot{\boldsymbol{q}}_d - \Lambda\dot{\widetilde{\boldsymbol{q}}}$ is the linear combination of velocity error and desired acceleration;
- Λ is a diagonal positive definite matrix.

The gains are chosen like this because we want to give more weight to the uncertain parameter, mainly the mass of the last link:

$$\begin{cases} \Lambda = \text{diag}[30, 30, 30, 30] \\ \boldsymbol{K}_D = \text{diag}[250, 250, 250, 250] \\ \boldsymbol{K}_\pi = \text{diag}[500, 500, 500, 500, 500, 500, 50, 500, 0.1, 500, 500, 500, 500, 500] \end{cases} \tag{50}$$

This choice for $\boldsymbol{K}_\pi$ allows to weigh more the uncertain parameter (the mass of the final link) in the adaptive law. With these parameters, we obtain these results:

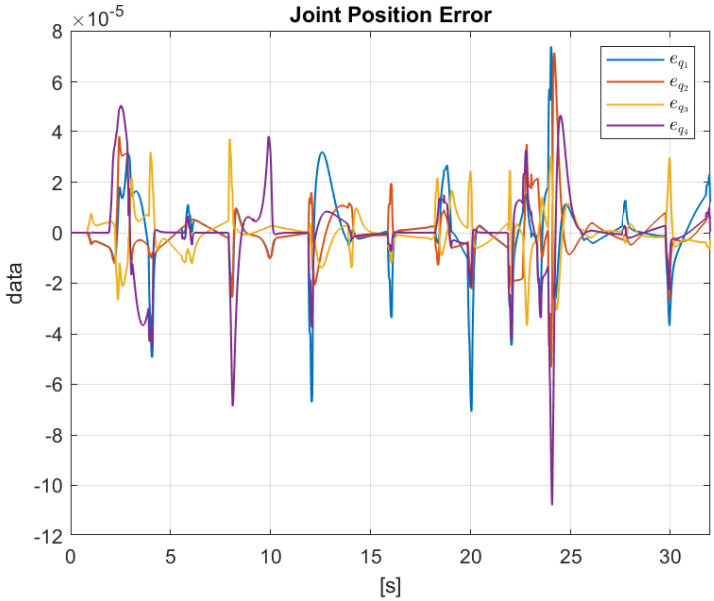

Figure 42: Joint position error.

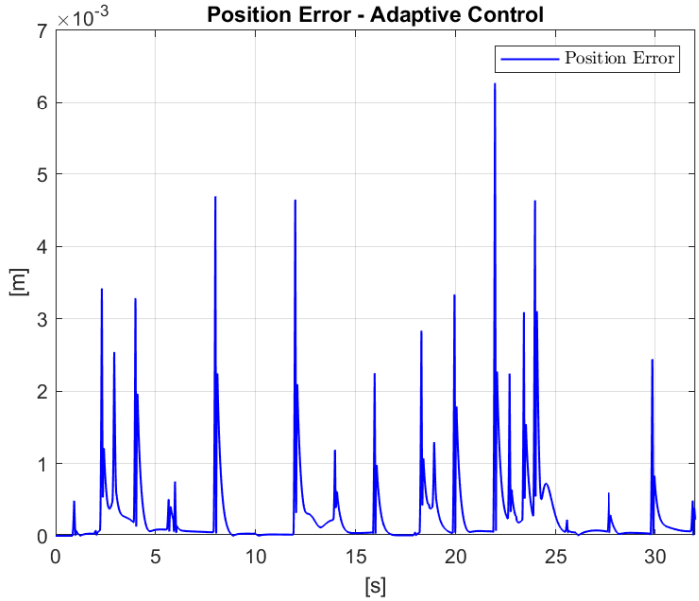

Figure 43: Norm of the end-effector position error.

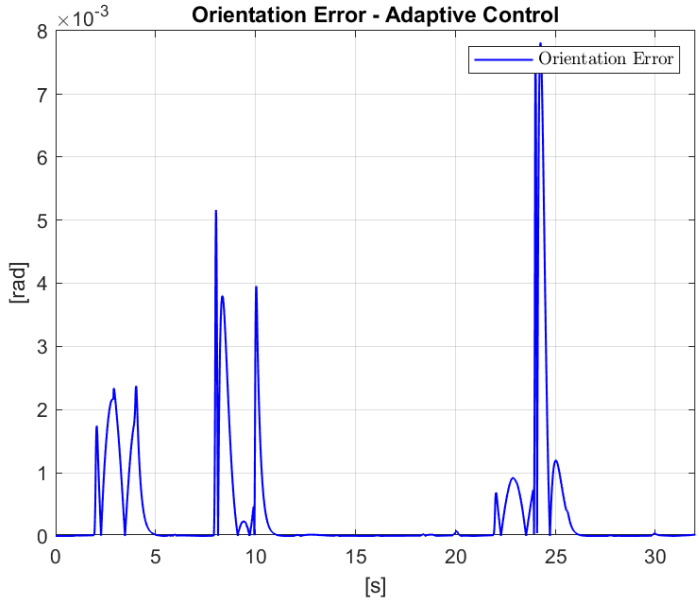

Figure 44: Norm of the end-effector orientation error.

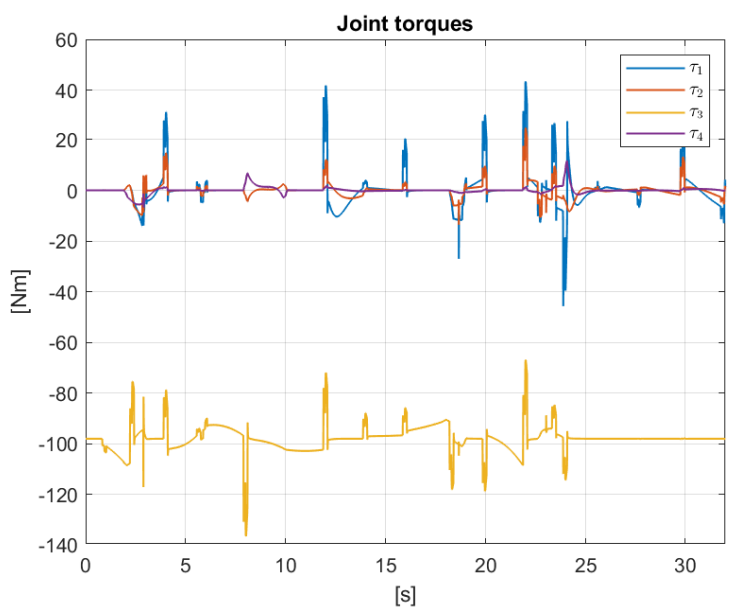

Figure 45: Joint torques.

References

[1] B. Siciliano, L. Sciavicco, L. Villani and G. Oriolo, *Robotics, Modelling, Planning and Control*, Advanced Textbooks in Control and Signal Processing, Springer, 2010.